CSS: Fonts, Text Properties, Shadows & Outline

A comprehensive lecture covering typeface control, text layout, visual effects, and accessibility-aware styling in CSS.

1

Fonts

family, size, weight, style

2

Text Properties

align, transform, spacing

3

Shadows

text-shadow, box-shadow

4

Outline

outline, offset, accessibility

1. CSS Fonts

Fonts control how text appears on a webpage. CSS provides a set of dedicated properties that give developers precise control over the visual presentation of every character rendered in the browser.

font-family

Specifies the typeface used to render text

font-size

Controls how large or small the text appears

font-style

Sets normal, italic, or oblique rendering

font-weight

Controls the boldness of the text

font-variant

Enables small-caps and other typographic variants

font-family

Specifies the typeface. The browser tries each font from left to right and uses the first one it finds installed on the system. Always end with a generic family as a safe fallback.

Single Font

```
body {
  font-family: Arial;
}
```

With Fallbacks

```
body {
  font-family: Arial, Helvetica, sans-serif;
}
```

Heading & Paragraph

```
h1 { font-family: Georgia, serif; }
p  { font-family: Arial, sans-serif; }
```

Generic Font Families

serif

Has decorative strokes at the ends of characters. Example: Georgia, Times New Roman.

sans-serif

Clean, modern style with no decorative strokes. Example: Arial, Helvetica.

monospace

Every character has equal width. Example: Courier New, Consolas.

cursive

Handwriting-like appearance. Example: Brush Script.

fantasy

Decorative, ornamental fonts used for display purposes.

2. font-size

Controls the size of text. CSS supports multiple unit types, each suited to different use cases — from fixed pixel sizes to scalable relative units.

Pixel Examples

```
h1 { font-size: 40px; }
p  { font-size: 18px; }
```

Relative Units

```
h1 { font-size: 2rem; }
p  { font-size: 18px; }
```

Common Units

px

Fixed size. Exact pixel value regardless of parent or root settings.

%

Percentage of the parent element's font size.

em

Relative to the font size of the parent element.

rem

Relative to the root element's font size. Consistent and predictable across the page.

3. font-weight

Controls how bold the text appears. You can use keyword values like `bold` or numeric values from 100 to 900 for finer control.

Keyword Value

```
p { font-weight: bold; }
```

Numeric Values

```
font-weight: 400; /* Normal */
font-weight: 500;
font-weight: 600;
font-weight: 700; /* Bold */
```

Full Numeric Scale

01

100 — Thin

Lightest possible weight, very delicate appearance.

02

400 — Normal

Default body text weight. Equivalent to the `normal` keyword.

03

700 — Bold

Standard bold. Equivalent to the `bold` keyword.

04

900 — Very Bold

Heaviest weight. Used for display headings and strong emphasis.

4. font-style

Controls whether text is rendered in a normal, italic, or oblique style. Italic is the most commonly used value, often applied to quotes, captions, and emphasis.

Basic Usage

```
p { font-style: italic; }
```

All Values

```
font-style: normal;
font-style: italic;
font-style: oblique;
```

Practical Example

HTML:

```
<p class="quote">
  Learning never stops.
</p>
```

CSS:

```
.quote {
  font-style: italic;
}
```

Use `italic` for semantic emphasis and quotes. Use `oblique` when a true italic variant of the font is not available — the browser will slant the normal style instead.

5. CSS Text Properties

Font properties control the appearance of individual characters. Text properties, by contrast, control how text is arranged, spaced, and displayed within its container. Both categories work together to produce polished typography.



color

Sets the text color



text-align

Horizontal alignment



text-decoration

Underline, overline, strikethrough



text-transform

Capitalization control



text-indent

First-line indentation



line-height

Vertical space between lines



letter-spacing

Space between characters



word-spacing

Space between words



text-shadow

Shadow effect behind text

6. text-align

Controls the horizontal alignment of text within its container. The `justify` value is particularly useful for long-form reading content such as articles and blog posts.

Basic Usage

```
h1 { text-align: center; }
```

All Values

```
text-align: left;
text-align: center;
text-align: right;
text-align: justify;
```

Example

```
.about {
  text-align: center;
}
```

The justify Value

Text is aligned to both the left and right edges of the container, creating clean, even margins on both sides — similar to how text appears in printed books and newspapers.

```
p {
  text-align: justify;
}
```

This is commonly used for longer paragraphs where a clean, uniform block of text is desired. Avoid it for short lines, as it can create awkward spacing between words.

7. text-decoration

Adds or removes decorative lines on text. One of its most common uses is removing the default underline from anchor tags to achieve a cleaner link style.

Basic Usage

```
a { text-decoration: underline; }
```

All Values

```
text-decoration: none;
text-decoration: underline;
text-decoration: overline;
text-decoration: line-through;
```

Strikethrough — Old Price

HTML:

```
<p class="old-price">₹999</p>
```

CSS:

```
.old-price {
  text-decoration: line-through;
}
```

Removing Link Underlines

```
a {
  text-decoration: none;
}
```

This removes the default underline from links — a very common pattern in modern web design.

8. text-transform

Changes the capitalization of text through CSS without altering the underlying HTML content. This is useful for enforcing consistent typographic style across headings, labels, and names.

Basic Usage

```
h1 { text-transform: uppercase; }
```

All Values

```
text-transform: uppercase;
text-transform: lowercase;
text-transform: capitalize;
```

Capitalize Example

HTML:

```
<p class="name">rahul kumar</p>
```

CSS:

```
.name {
  text-transform: capitalize;
}
```

Result

> **Rahul Kumar**

The `capitalize` value makes the first letter of each word uppercase, regardless of how the text is written in the HTML source.

9. text-indent

Adds indentation to the first line of a paragraph. This is a classic typographic technique used in article-style and editorial content to visually separate paragraphs without relying on extra margin or padding.

Basic Usage

```
p { text-indent: 40px; }
```

Article Example

```
.article {
  text-indent: 50px;
}
```

When to Use text-indent

Article Content

Indenting the first line of each paragraph is a traditional print convention that improves readability in long-form text.

Editorial Layouts

Useful for blog posts, news articles, and documentation where a clean, book-like appearance is desired.

Not for Short Text

Avoid using text-indent on headings, buttons, or short UI labels — it is intended for multi-line paragraph content.

10. line-height

Controls the vertical space between lines of text. A well-chosen line-height significantly improves readability, especially for longer paragraphs. It is one of the most impactful typographic properties for body text.

Recommended Usage

```
p { line-height: 1.6; }
```

Tight Spacing

```
p { line-height: 1; }
```

Loose Spacing

```
p { line-height: 2; }
```

Comparing Values

line-height: 1

Lines are tightly packed. Text can feel cramped and difficult to read, especially in long paragraphs.

line-height: 1.6

Comfortable reading experience. The recommended default for body text in most web projects.

line-height: 2

Very open spacing. Can feel airy and spacious — useful for certain design styles but may feel excessive for dense content.

A larger line-height generally makes long text easier to read.

11. letter-spacing

Controls the space between individual characters. Increasing letter-spacing is a common technique for headings, logos, and navigation labels to create a more refined, airy typographic feel.

Basic Usage

```
h1 { letter-spacing: 2px; }
```

Logo Example

```
.logo {
  letter-spacing: 5px;
}
```

12. word-spacing

Controls the space between words rather than individual characters. Useful for taglines and display text where a more open, spread-out appearance is desired.

```
p { word-spacing: 10px; }
```

```
.tagline {
  word-spacing: 8px;
}
```

letter-spacing affects space between every character. **word-spacing** only affects the space between words. Both accept positive and negative values.

13. Text Shorthand

Some font properties can be combined into a single `font` shorthand declaration. This reduces repetition and keeps stylesheets concise. However, beginners should first understand each individual property before using shorthand.

Longhand (Individual Properties)

```
h1 {
  font-style: italic;
  font-weight: bold;
  font-size: 30px;
  font-family: Arial;
}
```

Shorthand Equivalent

```
h1 {
  font: italic bold 30px Arial;
}
```

For beginners, first understand the individual properties before introducing shorthand. The shorthand order matters: `font-style font-weight font-size font-family`. Omitting required values can reset other font properties to their defaults unexpectedly.

14. Shadows

CSS provides two major shadow properties that add depth and dimension to web elements. Shadows are one of the most effective tools for creating modern, card-based UI designs.

text-shadow

Adds a shadow directly behind text characters. Used to create depth, glow effects, or improve text legibility against complex backgrounds.

```
h1 {
  text-shadow: 2px 2px 4px gray;
}
```

box-shadow

Creates a shadow around an element's entire box. One of the most commonly used CSS properties for cards, modals, buttons, and containers.

```
.card {
  box-shadow: 5px 5px 10px gray;
}
```

15. text-shadow

Adds a shadow behind text. The property accepts four values: horizontal offset, vertical offset, blur radius, and color. All four work together to define the shadow's position, softness, and appearance.

Basic Syntax

```
text-shadow: horizontal vertical blur color;
```

Example

```
h1 {
  text-shadow: 2px 2px 4px gray;
}
```

White Text with Dark Shadow

```
h1 {
  color: white;
  text-shadow: 2px 2px 5px black;
}
```

Breaking Down the Values

2px — Horizontal Position

Positive values move the shadow to the right. Negative values move it to the left.

2px — Vertical Position

Positive values move the shadow downward. Negative values move it upward.

4px — Blur Radius

Higher values produce a softer, more diffused shadow. Zero produces a hard-edged shadow.

---

gray — Shadow Color

Any valid CSS color value. Use `rgba()` for transparency control.

16. Multiple Text Shadows

CSS allows you to apply more than one shadow to the same text by separating each shadow declaration with a comma. This technique can create layered depth, glow effects, or embossed appearances.

Multiple Shadows Syntax

```
h1 {
  text-shadow:
    2px 2px 3px gray,
    -2px -2px 3px lightgray;
}
```

How It Works

Each comma-separated value is an independent shadow. The browser renders them in order, stacking them behind the text. In the example above:

- The first shadow (`2px 2px 3px gray`) falls to the bottom-right
- The second shadow (`-2px -2px 3px lightgray`) falls to the top-left

Together they create a subtle three-dimensional embossed effect.

For beginners, focus first on a single shadow. Multiple shadows are an advanced technique to introduce once the basic syntax is well understood.

17. box-shadow

Creates a shadow around an element's box. Although the lecture topic focuses on text shadows, `box-shadow` is one of the most commonly used CSS properties in modern web design and students should know it well.

Basic Example

```
.card {
  box-shadow: 5px 5px 10px gray;
}
```

Full Syntax

```
box-shadow: horizontal vertical blur spread color;
```

Common Card Style

```
.card {
  box-shadow: 0 4px 10px gray;
}
```

Modern Subtle Shadow

```
.card {
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.15);
}
```

Understanding the Parameters

horizontal

Offset along the X axis. Positive = right, negative = left.

vertical

Offset along the Y axis. Positive = down, negative = up.

blur

Softness of the shadow edge. Higher = more diffused.

spread

Expands or contracts the shadow size beyond the element's dimensions.

color

Use `rgba()` for semi-transparent shadows — the standard in modern UI design.

18. Outline

An outline is a line drawn *outside* the border of an element. Unlike borders, outlines do not take up space in the document layout — they do not affect the element's dimensions or surrounding elements.

Basic Usage

```
button {
  outline: 2px solid blue;
}
```

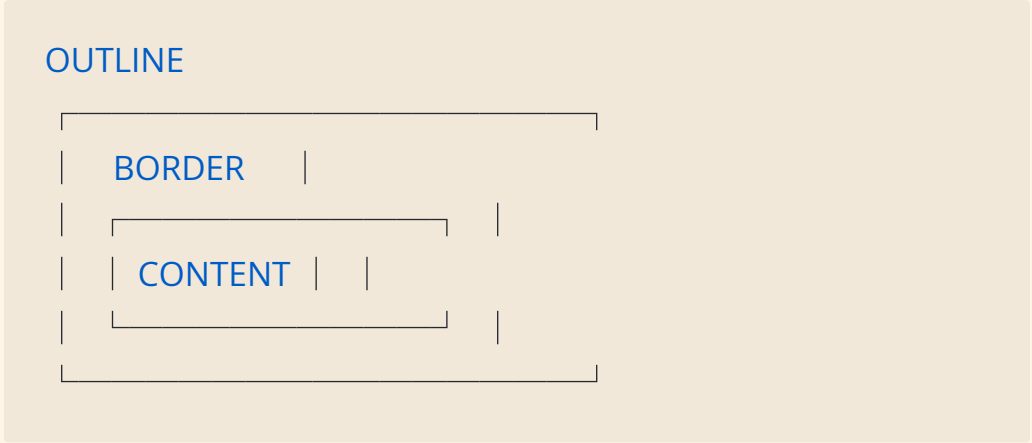
Border Declaration

```
border: 2px solid black;
```

Outline Declaration

```
outline: 2px solid red;
```

Border vs. Outline — Visual Model



Border

Sits inside the element's box model. Affects layout and takes up space.

Outline

Drawn outside the border. Does **not** affect layout or element dimensions.

19. Outline Properties

Just like borders, outlines have individual sub-properties for style, width, and color. These can be set individually or combined into a single shorthand declaration.

Individual Properties

```
outline-style: solid;
outline-width: 2px;
outline-color: red;
```

Shorthand

```
outline: 2px solid red;
```

outline-style Values

solid

A single continuous line. The most common value.

dashed

A series of short dashes.

dotted

A series of dots.

double

Two parallel lines.

none

Removes the outline entirely. Use with caution — see accessibility note.

20. outline-offset

`outline-offset` controls the distance between the element's border and its outline. This creates a visible gap between the two, which is a popular technique in modern focus indicator design.

Example

```
button {
  outline: 2px solid blue;
  outline-offset: 5px;
}
```

What outline-offset Does

Without `outline-offset`, the outline sits flush against the border. Adding a positive offset value pushes the outline outward, creating a gap between the element's border edge and the outline line.

offset: 0

Outline sits directly against the border with no gap.

offset: 5px

A 5px gap appears between the border and the outline — clean and modern.

Negative offset

Moves the outline inward, inside the element's border box.

21. Important: Outline and Accessibility

Students should **not** blindly remove outlines. Browsers use outlines to indicate which element currently has keyboard focus — a critical feature for users who navigate with a keyboard rather than a mouse.

❌ Do NOT do this blindly

```
outline: none;
```

✅ Provide a custom focus indicator instead

```
button:focus {
  outline: 3px solid blue;
  outline-offset: 3px;
}
```

Why This Matters

Keyboard Navigation

Users who cannot use a mouse rely on the Tab key to move between interactive elements. The outline shows them where focus currently is.

Screen Readers & Assistive Tech

Visible focus indicators are a core requirement of WCAG accessibility guidelines. Removing them creates barriers for users with disabilities.

The Right Approach

If the default browser outline looks unstyled, replace it with a custom styled outline — never simply remove it without providing an alternative.

22. Putting Everything Together — HTML

Now create a simple About Us section that applies the full range of properties covered in this lecture. The HTML structure below defines the content; the CSS on the next card applies all the styling.

```
<section class="about">
  <h1>About Us</h1>
  <p class="tagline">
    Building ideas. Creating experiences.
  </p>
  <p class="description">
    We are a team of passionate developers and designers who create
    modern and user-friendly digital experiences. Our goal is to turn
    ideas into useful and meaningful products.
  </p>
  <button class="btn">
    Learn More
  </button>
</section>
```

.about

The outer container — receives border, padding, background, and box-shadow

.tagline

Italic, bold, letter-spaced subtitle beneath the main heading

.description

Justified paragraph with controlled line-height for readability

.btn

Styled button with custom focus outline for accessibility

22. Putting Everything Together — CSS

This single CSS block covers almost the entire lecture. Study each rule and identify which property from the lecture it corresponds to.

```css
.about {
  width: 600px;
  padding: 40px;
  background-color: #f5f5f5;
  border: 2px solid #ddd;
  border-radius: 15px;
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.15);
  text-align: center;
}
.about h1 {
  font-family: Arial, sans-serif;
  font-size: 40px;
  font-weight: 700;
  color: #222;
  text-transform: uppercase;
  letter-spacing: 2px;
  text-shadow: 2px 2px 4px #ccc;
}
.tagline {
  font-size: 20px;
  font-weight: bold;
  font-style: italic;
  color: #555;
  letter-spacing: 1px;
}
.description {
  font-size: 16px;
  line-height: 1.7;
  text-align: justify;
  color: #444;
}
.btn {
  padding: 12px 25px;
  background-color: #222;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
}
.btn:focus {
  outline: 3px solid #555;
  outline-offset: 4px;
}
```

Properties Covered in This Example

Font

font-family · font-size · font-weight · font-style

Text

color · text-align · text-transform · letter-spacing · line-height · text-shadow

Box

box-shadow · border · border-radius · padding

Outline

outline · outline-offset (on `.btn:focus`)

Lab: About Us Page — Task 2

Objective

Create an About Us webpage for a fictional company, organization, college, startup, or development team. Use the CSS properties learned so far rather than copying a ready-made template.

Page Structure

```
------------------------------------------------
          OUR COMPANY
     "Building the future."
          About Us
------------------------------------------
  We are a team of passionate people...
  We create solutions that solve real-world
  problems and make technology accessible.

       OUR VALUES
  Innovation   Teamwork   Growth

         [ Learn More ]
------------------------------------------------
```

Suggested Challenge

Give students this requirement without giving them the CSS:

> Create an About Us page for a fictional technology company called **"CodeNova"**. The page should communicate who the company is, what it believes in, and what it does. Make the page visually attractive using only the CSS properties covered so far.

Requirements Checklist

- Main heading · Tagline · About paragraph · At least 3 values/features · Button
- Background color · Border · Border radius · Padding · Width and height where appropriate
- Different font families · Different font sizes · Font weight · Italic text
- Text alignment · Text transformation · Letter spacing · Line height
- Text shadow · Box shadow · Outline on an interactive element

Bonus Challenge

Ask students to create **three different visual styles** using the same HTML:

Professional Corporate

Conservative colors, serif fonts, formal spacing

Modern Startup

Bold sans-serif, vibrant accent colors, generous whitespace

Creative Developer Community

Dark theme, monospace fonts, glowing shadows

This is a good point to teach that CSS separates content from presentation: the HTML can remain almost identical while the entire visual appearance changes through CSS.